

PrepBuddy Adaptive PDF Preparation System

Technical Report

PrepBuddy Project

May 2026

Abstract

PrepBuddy is a local-first system for turning PDF documents into adaptive preparation sessions. It ingests PDFs, resolves document sections, stores a document-scoped knowledge base, generates grounded multiple-choice questions through hosted or local language models, scores answers, and adapts future sessions toward weak topics. The system exposes the same core workflow through a command-line interface, a FastAPI service, and a Streamlit user interface. This report describes the architecture, data model, PDF ingestion strategy, section mapping behavior, LLM provider design, adaptive session workflow, deployment modes, testing approach, and operational considerations.

Contents

1	Introduction	3
2	Goals and Requirements	3
2.1	Functional Goals	3
2.2	Nonfunctional Goals	3
3	System Overview	4
3.1	Major Components	4
4	Project Structure	4
5	PDF Ingestion and Section Mapping	5
5.1	Problem	5
5.2	Extraction Pipeline	5
5.3	Canonical IDs, Source Labels, and Aliases	5
5.4	Document-Scoped Mapping	6
6	Knowledge Base and Data Model	6
6.1	Core Tables	6
6.2	Archive Semantics	6
7	LLM Provider Layer	7
7.1	Provider Abstraction	7
7.2	Gemini Provider	7
7.3	Ollama Provider	7
7.4	Fake Provider	7
7.5	Validation and Repair	7

8	Adaptive Session Workflow	8
8.1	Session Generation	8
8.2	Cold Start	8
8.3	Returning Sessions	8
8.4	Scoring and Feedback	8
8.5	Knowledge Reset	8
9	User Interfaces	8
9.1	Streamlit UI	8
9.2	Command-Line Interface	9
9.3	REST API	9
10	Deployment	9
10.1	Windows	9
10.2	WSL and Linux	9
10.3	Docker	9
11	Testing and Quality Assurance	10
12	Security and Privacy Considerations	10
13	Operational Considerations	10
13.1	SQLite	10
13.2	Model Output Variability	10
13.3	PDF Quality	11
14	Limitations	11
15	Future Work	11
16	Conclusion	11

1 Introduction

Many study workflows start from long PDFs: reports, dossiers, textbooks, policy documents, lecture notes, or manuals. Turning those documents into useful practice material is usually repetitive. A user must identify the relevant sections, write questions, track performance, and decide what to revisit. PrepBuddy automates that workflow while keeping the document, history, and generated artifacts local by default.

The application is built around a simple idea: every generated question should be grounded in a selected part of a known document, and every future session should use the learner’s previous answers to spend more time on weaker areas. The result is not a generic quiz generator. It is a document-specific preparation system with persistent section mappings, session history, and adaptive feedback.

2 Goals and Requirements

2.1 Functional Goals

The system supports the following functional goals:

- Ingest a broad range of PDFs, including dossier-style documents, technical reports, and textbook-like files.
- Build a section map for each document and expose that mapping to the UI, CLI, and API.
- Generate multiple-choice questions from user-selected sections.
- Support a hosted LLM path through Gemini and a local LLM path through Ollama.
- Persist sessions, questions, answer choices, submitted answers, scoring results, and weak-topic statistics.
- Use prior attempts to adapt future prompts toward weak topics and away from repeated mastered questions.
- Provide a browser UI, terminal commands, and REST API endpoints over the same service layer.
- Preserve document history when a document is archived and reuse that history when the same PDF is uploaded again.

2.2 Nonfunctional Goals

The nonfunctional goals are:

- Local-first operation with SQLite as the default database.
- Predictable setup on Windows, WSL/Linux, and Docker.
- Clear failure modes for missing models, malformed model output, ambiguous section references, and missing document mappings.
- Deterministic testing through a fake LLM provider.
- Public, maintainable project structure using Python packaging conventions.

3 System Overview

PrepBuddy uses a backend-first architecture. The CLI, REST API, and Streamlit UI all call the same service layer. This keeps document ingestion, section mapping, generation, scoring, and persistence consistent regardless of entry point.

```
Streamlit UI ----\
prepbuddy CLI ----> PrepService ---> Repository ---> SQLite
FastAPI REST ----/      |
                        +--> PDF Ingestion + Section Mapping
                        |
                        +--> Adaptive Prompt Builder
                        |
                        +--> Gemini Provider
                        +--> Ollama Provider
                        +--> Fake Provider
```

3.1 Major Components

Component	Responsibility
CLI	Provides the <code>prepbuddy</code> terminal command for ingestion, sessions, exports, API launch, UI launch, and maintenance operations.
FastAPI service	Exposes document, section, session, answer submission, history, and maintenance endpoints.
Streamlit UI	Provides document upload, section selection, session generation, answer submission, history review, provider configuration, and maintenance actions.
PrepService	Coordinates ingestion, generation, scoring, adaptation, repository calls, and provider selection.
Repository	Encapsulates SQLAlchemy queries and persistence behavior.
PDF ingestion	Extracts text with PyMuPDF and builds top-level sections or page-range sections when headings are unreliable.
Mapping resolver	Assigns canonical IDs, source labels, titles, and unambiguous aliases for lookup.
Provider router	Selects Gemini, Ollama, or Fake generation based on configuration and availability.
SQLite database	Stores documents, sections, chunks, sessions, questions, answers, topic statistics, generation events, and snapshots.

4 Project Structure

The package uses a Python `src/` layout. The main package lives under `src/prepbuddy/`.

Path	Purpose
<code>src/prepbuddy/cli.py</code>	Typer command definitions and terminal workflows.
<code>src/prepbuddy/api.py</code>	FastAPI application and REST endpoints.

<code>src/prepbuddy/ui.py</code>	Streamlit application.
<code>src/prepbuddy/service.py</code>	Application orchestration and adaptive workflow.
<code>src/prepbuddy/repository.py</code>	Database operations and query behavior.
<code>src/prepbuddy/db.py</code>	SQLAlchemy models, engine setup, migrations, and SQLite pragmas.
<code>src/prepbuddy/ingestion.py</code>	PDF parsing and section extraction.
<code>src/prepbuddy/mapping.py</code>	Section mapping and alias normalization.
<code>src/prepbuddy/providers.py</code>	Gemini, Ollama, and fake provider implementations.
<code>src/prepbuddy/schemas.py</code>	Pydantic schemas and public data contracts.
<code>tests/</code>	Unit, integration, CLI, API, provider, UI helper, and Docker configuration tests.

5 PDF Ingestion and Section Mapping

5.1 Problem

PDFs are not structurally consistent. A clean dossier may contain explicit headings such as **Section 5. Tactics**. A technical report may use numbered headings such as **3.2 Methodology**. A textbook may repeat chapter titles in headers, list many numbered exercises, and contain references that look like headings. A robust preparation system cannot assume that every PDF uses sections 1 through 10.

5.2 Extraction Pipeline

PrepBuddy uses PyMuPDF to extract text page by page. The ingestion layer then attempts progressively broader detection strategies:

1. Explicit **Section N** headings.
2. Chapter-style headings.
3. Numeric headings.
4. Roman numeral headings.
5. Title-like top-level headings.
6. Stable page-range sections when heading detection is too noisy or ambiguous.

The fallback is important. It lets users prepare from ordinary books and reports instead of failing ingestion because a document has repeated labels or nonstandard headings.

5.3 Canonical IDs, Source Labels, and Aliases

Each stored section has:

- **Canonical ID**: the internal stable integer for selection and scenario commands.
- **Source label**: the label detected from the PDF, such as **Section 5, Chapter 2, or Pages 31–55**.

- **Title:** the detected or generated title.
- **Aliases:** normalized lookup names for safe resolution.

Aliases are useful, but only if they are unambiguous. The mapper therefore avoids accepting duplicate aliases that could resolve to multiple sections. When a repeated label appears in a report or textbook, the ingestion process should skip or qualify the ambiguous alias rather than reject the whole PDF.

5.4 Document-Scoped Mapping

Mappings are document-scoped. Section 5 in one PDF and section 5 in another PDF are separate entities. This prevents adaptive history from mixing unrelated documents that happen to use the same visible numbering.

6 Knowledge Base and Data Model

PrepBuddy stores local state in SQLite through SQLAlchemy. SQLite is a good fit because the application is single-user and local-first, while still providing transactions, indexes, foreign keys, and simple portability.

6.1 Core Tables

Table	Purpose
<code>documents</code>	One row per ingested PDF, including metadata, content hash, stored path, active/archive state, and upload information.
<code>sections</code>	Top-level document sections with canonical ID, label, title, page range, and extracted text.
<code>section_aliases</code>	Lookup aliases for section resolution.
<code>section_chunks</code>	Chunked section text used as LLM context.
<code>prep_sessions</code>	Generated or completed session metadata, provider information, score, and adaptation context.
<code>session_sections</code>	Join table linking sessions to selected sections.
<code>questions</code>	Generated MCQs, including topic, fingerprint, correct answer, and explanation.
<code>answer_choices</code>	Choice labels and text for each MCQ.
<code>answers</code>	Submitted answers and correctness results.
<code>topic_stats</code>	Aggregated attempts, correct counts, and wrong counts by document, section, and topic.
<code>generation_events</code>	Provider, model, latency, token usage, and warning metadata.
<code>kb_snapshots</code>	Persisted readable snapshots of recent knowledge-base state.
<code>app_state</code>	Application-level maintenance markers such as knowledge-base reset time.

6.2 Archive Semantics

Deleting a document from normal UI and CLI workflows archives it instead of hard-deleting its history. The document disappears from the active document list, but sessions, answers,

topic statistics, and mappings remain available for restoration. When the same PDF content is uploaded again, PrepBuddy uses the content hash to reactivate the existing document and reuse its history.

Hard deletion is reserved for explicit maintenance actions such as `clear-everything`.

7 LLM Provider Layer

7.1 Provider Abstraction

The provider layer defines a common interface for MCQ generation. This allows the service layer to validate and persist generated questions without caring whether they came from Gemini, Ollama, or a fake deterministic generator.

7.2 Gemini Provider

Gemini is the hosted fast path. It is selected automatically when a Gemini API key is present and provider mode is set to `auto`. The provider requests structured JSON and returns parsed MCQ data to the service layer.

7.3 Ollama Provider

Ollama is the local path. It lets the application run without a paid hosted API. The default local model is `qwen3:4b-instruct`. Output budgets are sized according to the requested question count so larger sessions are less likely to be truncated.

7.4 Fake Provider

The fake provider is deterministic. It exists for tests, demonstrations, and smoke runs where repeatability matters more than natural language quality.

7.5 Validation and Repair

LLM output is never accepted blindly. The service validates:

- exact total question count;
- exact per-section distribution;
- four choices per question;
- one valid correct answer;
- explanation presence;
- topic tag presence;
- valid section references;
- duplicate question fingerprints.

If a provider returns too few questions, the service performs bounded repair attempts for the missing sections. If the provider still cannot meet the request, the session fails clearly instead of silently showing fewer questions.

8 Adaptive Session Workflow

8.1 Session Generation

A session starts with a selected document, selected sections, provider mode, and question count per section. The service resolves each section against the selected document, fetches relevant chunks, looks up prior performance, and constructs a compact prompt.

8.2 Cold Start

When there is no useful history for the selected sections, PrepBuddy asks for balanced coverage across the selected material. The prompt emphasizes exact counts and grounded questions.

8.3 Returning Sessions

When history exists, the prompt includes:

- weak topics sorted by wrong answers and attempts;
- a short prior-session summary;
- recent question fingerprints to avoid repetition;
- selected section excerpts.

The goal is to spend more time on weak topics while still generating questions from the chosen document sections.

8.4 Scoring and Feedback

Submitted answers are scored against the persisted correct answer labels. Correct feedback is labeled as a correct answer. Wrong feedback is labeled as a wrong answer and includes the correct choice and a clarification based on the stored MCQ explanation.

8.5 Knowledge Reset

Clearing the knowledge base removes adaptive aggregates and records a reset marker. Existing sessions remain visible for review, but future adaptation ignores history before the reset timestamp.

9 User Interfaces

9.1 Streamlit UI

The UI is organized around three primary views:

- **Start:** document table, document selection, section map, section selector, question count, and session generation.
- **Session:** numbered MCQs, answer submission, score, and feedback.
- **Knowledge:** readable tables for sessions, missed questions, and weak topics.

The sidebar contains upload controls, document rows with delete actions, document-scoped session history, provider settings, optional Gemini key entry, and maintenance actions.

9.2 Command-Line Interface

The CLI is implemented with Typer and Rich. It supports:

- environment and path checks through `prepbuddy doctor`;
- document ingestion and listing;
- section and mapping inspection;
- session generation and simulated or interactive answers;
- scenario exports;
- knowledge-base snapshots;
- maintenance commands for sessions, documents, and adaptive data;
- launching the API and UI.

9.3 REST API

The FastAPI app exposes document, section, session, answer submission, knowledge snapshot, and maintenance endpoints. It uses the same service layer as the CLI and UI, so behavior is consistent across interfaces.

10 Deployment

10.1 Windows

The Windows path uses a virtual environment named `venv`. The installed console script is available as `prepbuddy` inside the activated environment. Streamlit can open the browser automatically on normal desktop Windows installations.

10.2 WSL and Linux

The WSL/Linux path uses a virtual environment named `projectenv`. Browser launching may depend on WSL desktop integration. If automatic launch is unavailable, the UI command prints a local URL that can be opened manually.

10.3 Docker

The Docker setup provides:

- an API service on port 8000;
- an optional UI service on port 8501;
- mounted local directories for `data/`, `config/`, `outputs/`, and `docs/`;
- optional containerized Ollama through a compose profile.

Containers do not attempt to open the host browser. Users open the printed host URL directly.

11 Testing and Quality Assurance

The project uses pytest and Ruff. The test suite covers:

- direct section numbering, report-like headings, textbook-like fallbacks, and ambiguous aliases;
- repository behavior for multiple documents, archives, reactivation, sessions, and snapshots;
- provider validation, exact-count repair, and Ollama error handling;
- CLI document/session maintenance;
- API document upload, mapping, session creation, answer submission, and deletion flows;
- UI helper formatting for question numbering, score display, key masking, and readable snapshots;
- Dockerfile and compose configuration.

The fake provider enables deterministic tests without requiring external network calls or local model availability.

12 Security and Privacy Considerations

PrepBuddy is local-first, but users can choose hosted inference. The privacy model depends on provider selection:

- With Ollama or the fake provider, generation stays local.
- With Gemini, selected document excerpts are sent to the hosted API.
- Gemini keys are never printed in full and can be kept session-only in the UI.
- Uploaded PDFs are stored under the local data directory when managed by the app.
- The app does not implement multi-user authentication and should not be exposed directly to untrusted networks.

13 Operational Considerations

13.1 SQLite

SQLite keeps setup simple. It is suitable for local single-user workflows and tests. If PrepBuddy were extended into a multi-user hosted service, the repository layer would be the natural boundary for moving to PostgreSQL or another server database.

13.2 Model Output Variability

Structured output reduces parsing failures, but language models can still under-generate, repeat questions, or ignore requested distribution. PrepBuddy handles this through validation, bounded repair, and clear provider errors.

13.3 PDF Quality

PDF extraction quality depends on the source file. Native text PDFs work best. Scanned documents require OCR before ingestion. Highly decorative layouts may produce page-range sections rather than semantic chapter sections.

14 Limitations

- PrepBuddy does not perform OCR.
- It does not include authentication or multi-user permissions.
- Question quality depends on extracted text quality and model behavior.
- Page-range fallback is intentionally conservative and may be less semantically precise than manual section mapping.
- Hosted inference may send selected excerpts to the configured provider.

15 Future Work

Useful future improvements include:

- optional OCR for scanned PDFs;
- richer manual section editing in the UI;
- semantic search over large document libraries;
- additional question types beyond MCQs;
- export formats for Anki, CSV, and learning management systems;
- a multi-user deployment mode with authentication and authorization.

16 Conclusion

PrepBuddy provides a practical local system for document-grounded study. Its main design choices are conservative: a shared service layer, document-scoped history, robust section mapping, provider-independent validation, SQLite persistence, and deterministic tests. These choices make the app easy to run on a laptop while leaving clear extension points for larger deployments.